MINI PROJECT REPORT ON

"Expense Tracker"

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS OF DEGREE OF

BACHELOR OF ENGINEERING BY

Tejas Mandhyani (Roll No. 2501135)

Bhoomi Ramrakhiani (Roll No. 2301095)

Jainil Shah (Roll No. 2301106)

SUBJECT

Agile Software Development and DevOps

SUPERVISOR

Prof. Sanober Shaikh



DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE
THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050

UNIVERSITY OF MUMBAI [Academic Year: 2026–27]

THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050 A.Y. - 2026-27

Certificate

This is to certify that the Mini Project entitled **"Expense Tracker"** is a bonafide work of **Tejas Mandhyani, Bhoomi Ramrakhiani and Jainil Shah** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **"Undergraduate"** in **"Artificial Intelligence and Data Science".**

Prof. Sanober Shaikh
(Supervisor)

Abstract

The Expense Tracker is a web application that lets users record their income and expenses, organise them by category and review their spending in one place. The application runs as a containerised web service backed by a MySQL 8.0 database, so that all transactions are stored persistently and can be queried at any time.

Beyond the application, the project demonstrates a practical DevOps and agile workflow. The whole stack is packaged with Docker and started together with Docker Compose as four services: the web application, the MySQL database, Prometheus and Grafana Alloy. Prometheus collects metrics and Grafana Alloy forwards telemetry to Grafana, where dashboards are built. Source code is managed on GitHub using a feature-branch workflow (feature/grafana-monitoring), and the work is planned and tracked on a Jira Scrum board in the project “Expense Tracker DevOps” (issues ETD-1 to ETD-4).

TABLE OF CONTENTS

Content **Page No.**

Abstract **3**

List of Figures **4**

1. Introduction **5**

1.1 Background 5

1.2 Problem Statement 5

2. Implementation **6**

2.1 Technologies Used 6

2.2 Code Structure 6

2.3 Database Design 7

2.4 Expense Tracking Logic 7

2.5 Monitoring and Delivery Workflow 8

3. Results and Discussion **9**

3.1 Result 9

3.2 Discussion 10

4. Conclusion **11**

5. References **12**

LIST OF FIGURES

Fig. 3.1 Docker Desktop showing the Expense Tracker Compose services running 9

Fig. 3.2 GitHub repository branches for Expense-Tracker-ASD-D 9

Fig. 3.3 Jira Scrum board for the Expense Tracker DevOps project 10

1. Introduction

1.1 Background

Managing personal money is difficult without a clear record of where it goes. Many people note expenses on paper or remember them loosely, which makes it hard to see how much was spent on food, travel or bills, or to stay within a monthly budget. A simple digital tracker solves this by storing each transaction with its amount, category and date and by summarising the data.

The Expense Tracker project provides such a web application. Users can add, view and manage their expenses through a browser, and the data is stored in a MySQL database. The application is served on port 8080 from its own container.

The project also includes a DevOps layer: Docker and Docker Compose for packaging, Prometheus and Grafana for monitoring, GitHub for version control and Jira for agile sprint planning. Together these show how an application can be built, run, observed and managed in a repeatable way.

1.2 Problem Statement

The problem is the lack of an easy, reliable way to record and review everyday expenses, together with the effort needed to set up, deploy and monitor such an application consistently on different machines. Manual setup is slow and error-prone, and without monitoring the team has no visibility into how the application and its database behave.

The system therefore aims to provide the following capabilities:

1. Let users record expenses and income with an amount, category and date.
2. Store all records persistently in a MySQL database and allow them to be viewed and managed.
3. Package the web application and the database as containers so that they run the same way on any machine.
4. Start the complete stack, including the monitoring services, with a single Docker Compose command.
5. Collect metrics with Prometheus and Grafana Alloy and display them on Grafana dashboards.
6. Manage the source code on GitHub using feature branches and plan the work on a Jira Scrum board.

Chapter 2

Implementation

2.1 Technologies Used

Component	Technology	Port	Role
Web application	Expense Tracker image (container web-1)	8080:80	User interface and application logic
Database	MySQL 8.0 (container db-1)	Internal	Persistent storage of expenses
Prometheus	prom/prometheus (container prometheus-1)	9090:9090	Metric scraping and time-series storage
Grafana Alloy	grafana/alloy (container alloy-1)	12345:12345	Telemetry collector forwarding data to Grafana
Grafana	Grafana dashboards	Cloud / local	Visualisation of metrics
Version control	Git and GitHub	None	Source code and branch management
Issue tracking	Jira (project key ETD)	None	Scrum board and sprint tracking

Table 2.1: Technology stack and ports (as mapped in Docker Desktop)

- **User interface:** the web application is served by the web-1 container and is reached in the browser on port 8080.
- **Application logic:** the application receives requests to add, list and manage expenses and reads and writes them in the database.
- **Database:** a MySQL 8.0 container (db-1) stores all transactions persistently.
- **Observability:** Prometheus scrapes metrics on port 9090, and Grafana Alloy (port 12345) collects and forwards telemetry to Grafana.
- **Containerisation:** Docker packages each service, and Docker Compose runs them together as the project expense_tracker.
- **Version control and tracking:** Git and GitHub manage the code, and the Jira board “Expense Tracker DevOps” tracks the tasks ETD-1 to ETD-4.

2.2 Code Structure

The repository (Expense-Tracker-ASD-D) is organised so that application code and infrastructure files are kept separate.

- **Application source:** the Expense Tracker web application code and its Dockerfile.
- **docker-compose.yml:** defines the web, db, prometheus and alloy services, their port mappings and the shared network.

- **Monitoring configuration:** the Prometheus configuration file that lists the scrape targets, and the Gra
- **Branches:** main is the default branch, and the feature/grafana-monitoring branch holds the monitoring work before it is merged.

2.3 Database Design

The application uses a relational MySQL 8.0 database. The core entity is the expense (transaction) record. A typical design is shown below.

Field	Type	Description
id	INT (primary key)	Unique identifier of the record
title / description	VARCHAR	Short description of the expense
amount	DECIMAL	Amount spent or earned
category	VARCHAR	Category such as food, travel or bills
date	DATE	Date of the transaction

Table 2.2: Expense record structure

The database runs in its own container and stores data separately from the web application, so the application can be rebuilt or restarted without losing the records.

2.4 Expense Tracking Logic

The core workflow of the application is as follows:

1. The user opens the application in the browser and enters the details of a new expense.
2. The web application validates the input and saves the record in the MySQL database.
3. The saved expenses are read back from the database and shown to the user as a list.
4. The user can review the records by category and date to understand where the money is being spent.

Because the data lives in the database and not in the browser, the records remain available across sessions and devices.

2.5 Monitoring and Delivery Workflow

Monitoring is added as a separate branch of work (ETD-4: Set up Grafana monitoring and dashboards). Prometheus scrapes metrics from its targets and stores them as time series. Grafana Alloy runs as a collector and forwards the telemetry to Grafana, where dashboards can be created to view the health of the services.

The delivery workflow combines the agile and DevOps tools used in the project:

Step	What it does
1. Plan in Jira	Tasks ETD-1 to ETD-4 are created on the Scrum board and moved through To Do, In Progress and Done.
2. Branch in GitHub	Work for a task is done on a feature branch, for example feature/grafana-monitoring.
3. Containerise	The application is packaged with Docker and defined in docker-compose.yml.
4. Run the stack	docker compose up -d starts the web, db, prometheus and alloy containers together.
5. Monitor	Prometheus and Grafana Alloy collect metrics that are shown in Grafana.

Table 2.3: Delivery workflow

Chapter 3

Results and Discussion

3.1 Result

- **Application:** the Expense Tracker web application runs in the web-1 container and is available on port 8080.
- **Database:** the MySQL 8.0 container (db-1) runs alongside the application and stores the expense data.
- **Monitoring:** Prometheus is exposed on port 9090 and Grafana Alloy on port 12345, and both containers show a healthy running status in Docker Desktop.
- **Containerisation:** all services start together under the Compose project expense_tracker with one command.
- **Version control:** the GitHub repository Expense-Tracker-ASD-D has the main branch and the feature/grafana-monitoring branch, which is one commit ahead of main.
- **Agile tracking:** the Jira board shows ETD-1 (GitHub repository and version control) and ETD-2 (containerise using Docker) in Done, ETD-3 (Jira Scrum board) in progress and ETD-4 (Grafana monitoring) in To Do.

Screenshots



Fig. 3.1: Docker Desktop showing the Expense Tracker Compose services running

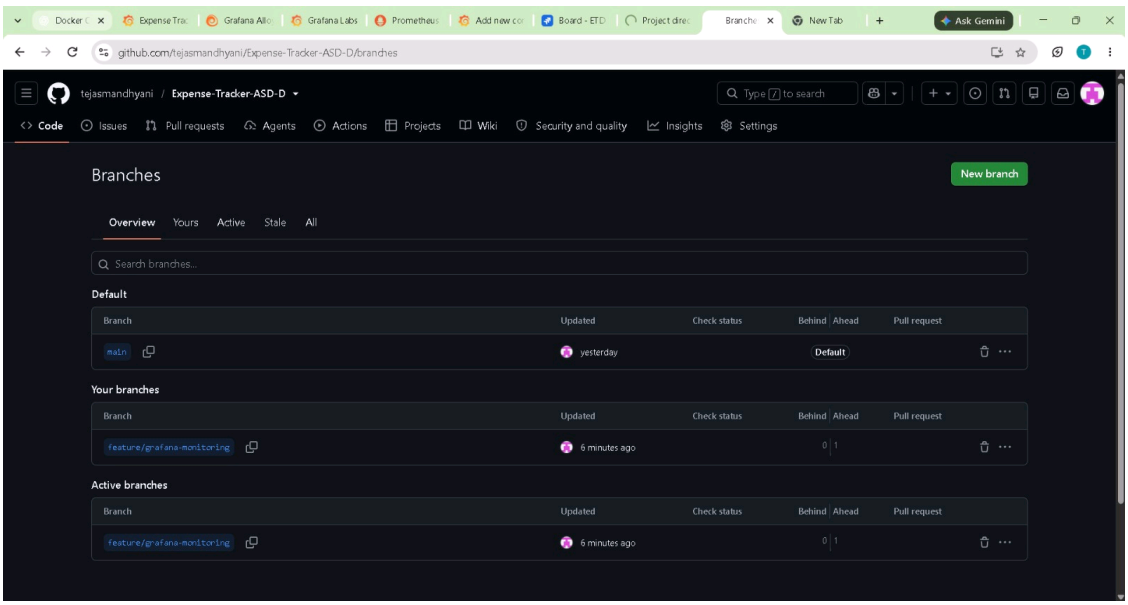


Fig. 3.2: GitHub repository branches for Expense-Tracker-ASD-D

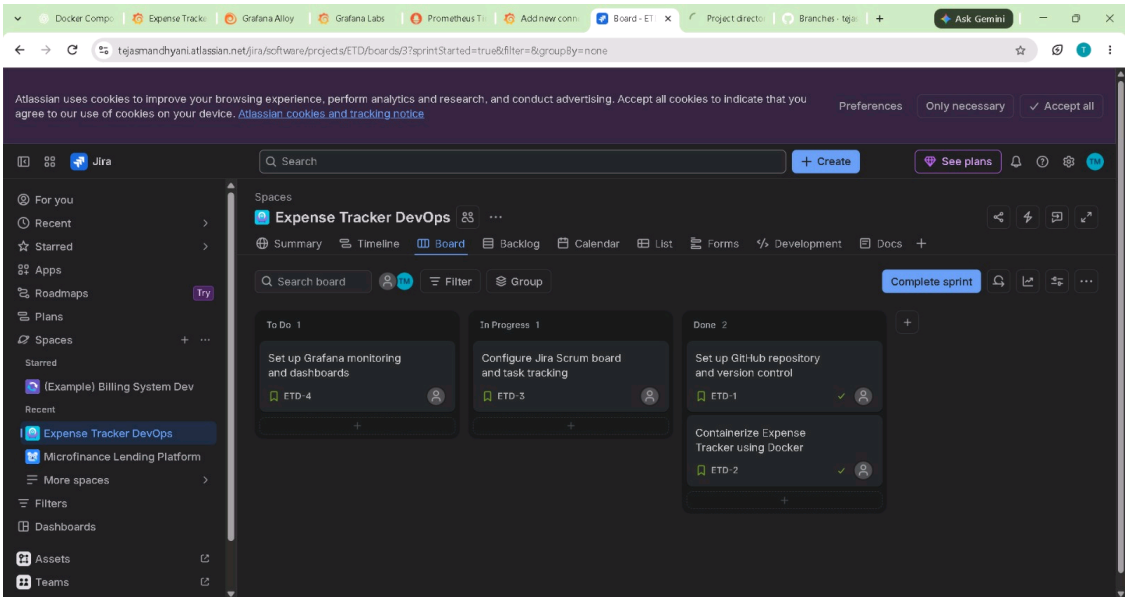


Fig. 3.3: Jira Scrum board for the Expense Tracker DevOps project

3.2 Discussion

The results confirm that the project meets its goals. Packaging the application and its MySQL database as containers removes the need for manual installation, and Docker Compose lets the whole stack, including Prometheus and Grafana Alloy, be started with a single command. This makes the environment repeatable for every team member.

Using Jira with a Scrum board gave the team a clear view of progress, since every task from setting up the repository to adding Grafana dashboards is a visible work item. Working on a separate feature branch in GitHub kept the monitoring work isolated from the stable main branch until it was ready to merge.

Identified Limitations

- **No authentication:** the prototype does not yet have user login, so all users share the same data. A rea
- **Limited analytics:** expenses can be recorded and listed, but advanced reports such as monthly budgets, charts and spending alerts are not yet available.
- **Manual deployment:** there is no automated CI/CD pipeline yet, so builds and deployments are started manually.

Chapter 4

Conclusion

4.1 Conclusion

The Expense Tracker demonstrates how a simple and useful application can be built and delivered using modern agile and DevOps practices. The web application and its MySQL database are packaged with Docker and run together with Prometheus and Grafana Alloy using Docker Compose. Source code is managed on GitHub with feature branches, and the work is planned and tracked on a Jira Scrum board.

The project shows how containerisation, monitoring, version control and sprint planning work together to make development repeatable, transparent and easier to maintain, fulfilling the requirements of the Agile Software Development and DevOps curriculum.

4.2 Future Scope

- **User authentication:** add secure login and per-user data using JWT-based authentication.
- **Reports and charts:** add monthly budgets, category-wise charts and spending alerts.
- **CI/CD pipeline:** add a Jenkins or GitHub Actions pipeline to run tests and build images automatically on every push.
- **Custom application metrics:** export business metrics such as expenses added per day and view them on Grafana dashboards.
- **Cloud deployment:** deploy the stack to a cloud server or Kubernetes cluster with automated backups of the database.

Chapter 5

References

1. **Docker Documentation** – Containerization and Docker Compose reference, https://docs.docker.com/
2. **MySQL 8.0 Reference Manual** – https://dev.mysql.com/doc/refman/8.0/en/
3. **Prometheus Documentation** – Monitoring system and time series database, https://prometheus.io/docs/
4. **Grafana Documentation** – The open observability and visualization platform, https://grafana.com/docs/
5. **Grafana Alloy Documentation** – OpenTelemetry Collector distribution, https://grafana.com/docs/alloy/latest/
6. **GitHub Docs** – Version control and branching, https://docs.github.com/
7. **Atlassian Jira Software Documentation** – Scrum boards and sprints, https://support.atlassian.com/jira-software-cloud/